

```
[ ]  
#lab 15  
  
import torch  
from PIL import Image  
import matplotlib.pyplot as plt  
import urllib.request  
import os  
  
# Step 1: Automatically download a sample image  
img_path = "street.jpg"  
if not os.path.exists(img_path):  
    url = "https://ultralytics.com/images/zidane.jpg" # sample image  
    urllib.request.urlretrieve(url, img_path)  
    print(" Sample image downloaded successfully: street.jpg")  
else:  
    print(" Image already exists:", img_path)  
  
# Step 2: Load the pre-trained YOLOv5 model  
# This will download YOLOv5 automatically from Ultralytics (first time only)
```


Follow link (ctrl + click)



9.ipynb

File Edit View Insert Runtime Tools Help

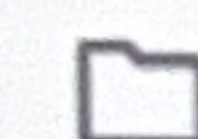
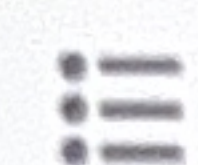


Share

N

Commands + Code + Text Run all

Reconnect



```
detected_img_path = os.path.join(output_folder, os.path.basename(img_path))
if os.path.exists(detected_img_path):
    detected_img = Image.open(detected_img_path)
    plt.imshow(detected_img)
    plt.axis('off')
    plt.title("YOLOv5 Object Detection Result")
    plt.show()
```



Image already exists: street.jpg

Loading YOLOv5 model (this may take a few seconds)...
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com>
Using cache found in /root/.cache/torch/hub/ultralytics_yolov5_master
YOLOv5 2025-11-1 Python-3.12.12 torch-2.8.0+cu126 CPU

Variables

Terminal



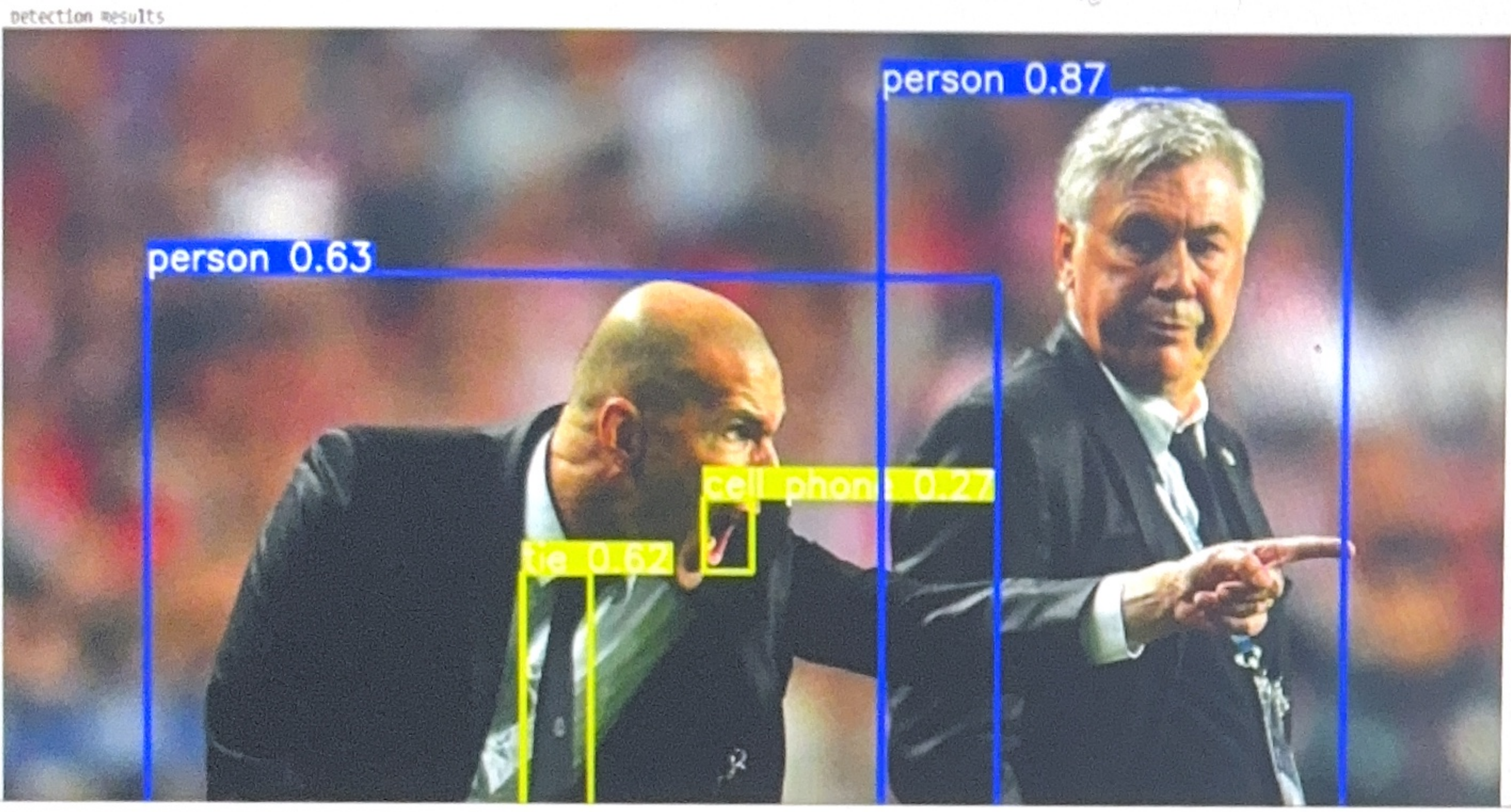
9:19 AM


```
Image already exists: street.jpg

Loading YOLOv5 model (this may take a few seconds)...
Creating new ultralytics settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at '/root/.config/ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Using cache found in /root/.cache/torch/hub/ultralytics_yolov5_master
YOLOv5 2025-11-1 Python-3.12.12 torch-2.8.0+cu126 CPU

Downloading https://github.com/ultralytics/yolov5/releases/download/v7.0/yolov5s.pt to yolov5s.pt...
100% 14.1M/14.1M [00:00:00, 114MB/s]

Fusing layers...
YOLOv5s summary: 213 layers, 7225685 parameters, 0 gradients, 16.4 GFLOPs
Adding AutoShape...
/root/.cache/torch/hub/ultralytics_yolov5_master/models/common.py:906: FutureWarning: 'torch.cuda.amp.autocast(args...)' is deprecated. Please use 'torch.amp.autocast('cuda', args...)' instead.
  with amp.autocast(auto):
Image 1/1: 1 phone
Speed: 89... 2, 45.7ms NMS per image at shape (1, 3, 384, 640)
```



15. Implement a YOLO model to detect Object

Aim:- To implement YOLO model for real time object detection and understand its working principle.

Objectives:-

- To study how YOLO detects multiple objects in single image
- To understand the architecture and working of pre-trained YOLO model.
- To use transfer learning for custom object detection.
- To visualize the bounding boxes and class labels predicted by YOLO.

Pseudo code :-

1. Install and import ultralytics.
2. load a pre-trained YOLO model.
3. load a test image or use a camera frame
4. Run the model. predict () method to detect objects.
5. displays bounding boxes and class labels.
6. Save the annotated output image.

Observations:-

1. The pre-trained YOLOv5s model successfully detected multiple objects such as cars, buses & persons in a single frame
2. Each object was enclosed in a bounding box with a class label and confidence score.
3. The inference time per image was very low, showing

YOLO's high efficiency for real time use.

4. The model demonstrated strong generalization without additional training.
5. The visualization clearly showed YOLO's ability to detect overlapping objects in complex scenes.

Result:-

successfully implemented a pre-trained YOLO model.

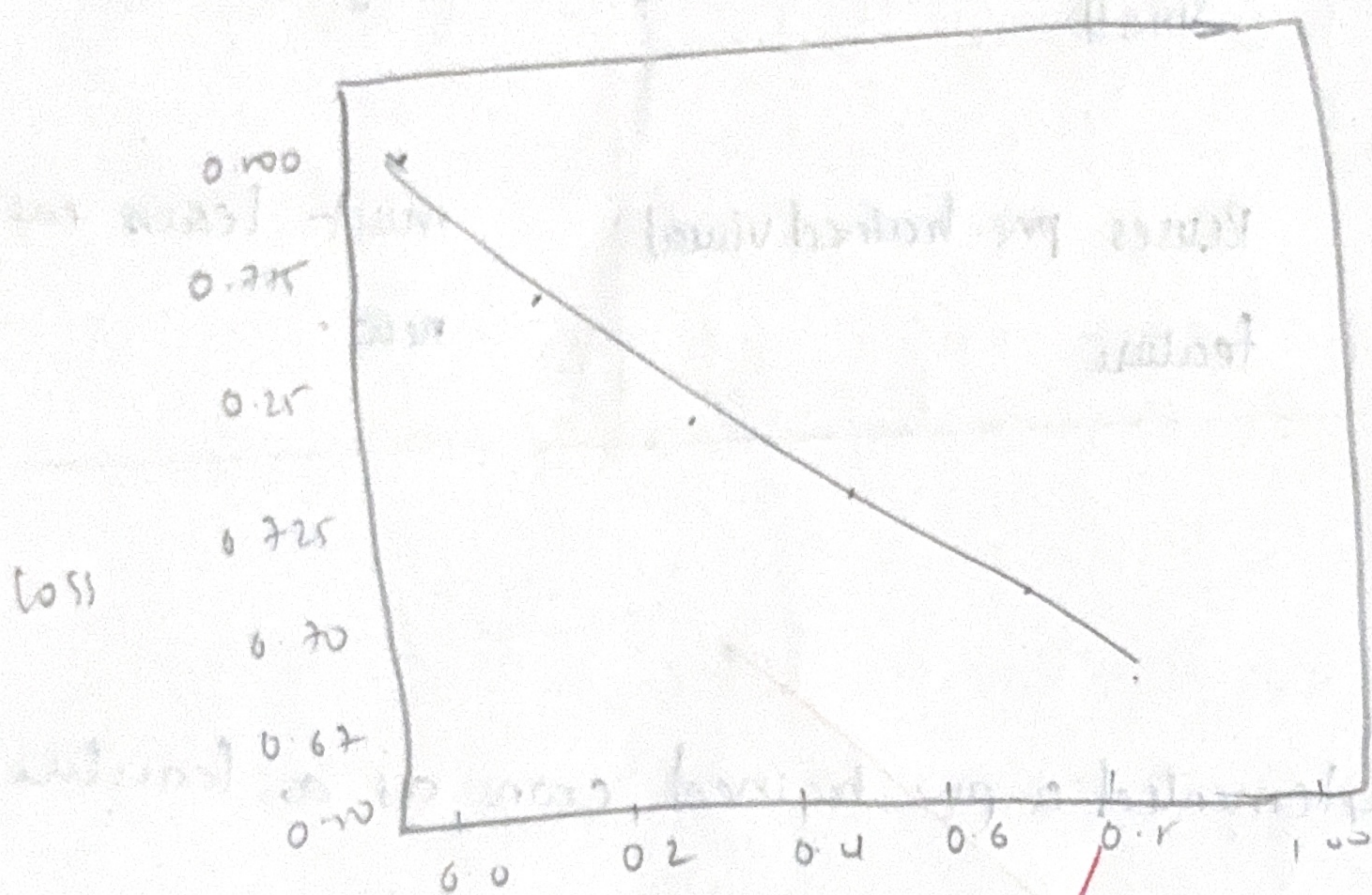
~~Ques 1~~

Output

Epoch (1/2) loss: 0.817

Epoch (2/2) ; loss ; 0.6423

Training loss per epoch.



Epoch

Test accuracy : 79.25%

Implementation of YOLO model to detect Object

Output:-

Image 1/1: 420 x 1280 2 person, 1 lie, 1 call phone

Speed 89-3ms p. upon C430, 122 qms, inference 45.7ms, 2ms, Nms, Objectives-

pre image ut shape (1, 3, 420, 640)

- To use transfer learning for custom object detection.
- To visualize the bounding boxes and class labels predicted by YOLO.

Pseudo code:-

1. Install and import libraries.
2. Load a pre-trained YOLO model.
3. Load a test image or use a camera frame.
4. Run the model's predict() method to detect objects.
5. Display bounding boxes and class labels.
6. Save the annotated output image.

Observations:-

1. The pre-trained YOLO model successfully detected multiple